



Exercícios : Gestão de tarefas

Professor: Rogério Pereira Junior

rogerio.pereira@ifsc.edu.br

Baseado no conteúdo do livro “Sistemas Operacionais: Conceitos e Mecanismos” do Prof. Carlos A. Maziero (UFPR)

1. Diferenciação Conceitual: Explique a diferença entre um "programa" e uma "tarefa" (ou processo). Utilize uma analogia para expressar suas idéias.
2. Descreva os cinco estados principais de uma tarefa em um sistema de tempo compartilhado (Nova, Pronta, Executando, Suspensa e Terminada). O que caracteriza a transição de "Executando" para "Pronta"?
3. O que significa time sharing e qual a sua importância em um sistema operacional?
4. Defina o conceito de processo.
5. Por que um processo é atualmente considerado um "contêiner de recursos" em vez de apenas uma unidade de execução? O que as threads dentro de um mesmo processo compartilham entre si?
6. Indique se cada uma das transições de estado de tarefas a seguir definidas é possível ou não. Se a transição for possível, dê um exemplo de situação na qual ela ocorre (N: Nova, P: pronta, E: executando, S: suspensa, T: terminada).

• $E \rightarrow P$

• $E \rightarrow S$

• $S \rightarrow E$

• $P \rightarrow N$

• $S \rightarrow T$

• $E \rightarrow T$

• $N \rightarrow S$

• $P \rightarrow S$

7. Relacione as afirmações abaixo aos respectivos estados no ciclo de vida das tarefas (N: Nova, P: Pronta, E: Executando, S: Suspensa, T: Terminada):

- [] O código da tarefa está sendo carregado.
- [] As tarefas são ordenadas por prioridades.
- [] A tarefa sai deste estado ao solicitar uma operação de entrada/saída.
- [] Os recursos usados pela tarefa são devolvidos ao sistema.
- [] A tarefa vai a este estado ao terminar seu quantum.
- [] A tarefa só precisa do processador para poder executar.
- [] O acesso a um semáforo em uso pode levar a tarefa a este estado.
- [] A tarefa pode criar novas tarefas.
- [] Há uma tarefa neste estado para cada processador do sistema.
- [] A tarefa aguarda a ocorrência de um evento externo.

8. Considere um sistema operacional multitarefa e um processo contendo múltiplas threads.

Assinale a alternativa correta:

- (a) Cada thread possui seu próprio espaço de endereçamento.
- (b) Threads de um mesmo processo compartilham arquivos e memória.
- (c) Cada thread corresponde necessariamente a um processo.
- (d) Threads não possuem contador de programa.

9. Sobre os modelos de implementação de threads, analise as afirmativas:

- I. No modelo N:1, chamadas bloqueantes afetam todas as threads.
- II. No modelo 1:1, cada thread pode ser escalonada independentemente.
- III. O modelo N:M elimina completamente a necessidade de threads de núcleo.

Assinale a alternativa correta:

- (a) Apenas I
- (b) Apenas I e II
- (c) Apenas II e III
- (d) I, II e III

10. Explique o que é, para que serve e o que contém um TCB - Task Control Block.

11. Desenhe o diagrama de tempo da execução do código a seguir, informe qual a saída do programa na tela (com os valores de x) e calcule a duração aproximada de sua execução.

```
1 #include <stdio.h>
2 #include <unistd.h> // para fork() e sleep()
3 #include <sys/wait.h> // para wait()
4 int main() {
5     int x = 0;
6     fork();
7     x++;
8     sleep(5);
9     wait(0);
10    fork();
11    wait(0);
12    sleep(5);
13    x++;
14    printf("Valor de x: %d\n", x);
15    return 0;
16 }
```

12. O que são threads e para que servem?
13. Quais as principais vantagens e desvantagens de threads em relação a processos?
14. A tabela a seguir representa um conjunto de tarefas prontas para utilizar um processador: Represente graficamente a sequência de execução das tarefas e calcule os tempos médios

Tarefa	t_1	t_2	t_3	t_4	t_5
ingresso	0	0	3	5	7
duração	5	4	5	6	4
prioridade	2	3	5	9	6

de vida (turnaround time) e de espera (waiting time), para as políticas de escalonamento a seguir:

- a) FCFS cooperativa
- b) SJF cooperativa
- c) SJF preemptiva (SRTF)
- d) PRIO cooperativa
- e) PRIO preemptiva
- f) RR com $t_q = 2$, sem envelhecimento

Considerações: todas as tarefas são orientadas a processamento; as trocas de contexto têm duração nula; em eventuais empates (idade, prioridade, duração, etc), a tarefa t_i com menor i prevalece; valores maiores de prioridade indicam maior prioridade.

15. Associe as afirmações a seguir aos seguintes modelos de threads: a) many-to-one (N:1); b) one-to-one (1:1); c) many-to-many (N:M):
 - (a) Tem a implementação mais simples, leve e eficiente
 - (b) Multiplexa os threads de usuário em um pool de threads de núcleo.
 - (c) Pode impor uma carga muito pesada ao núcleo.
 - (d) Não permite explorar a presença de várias CPUs pelo mesmo processo.
 - (e) Permite uma maior concorrência sem impor muita carga ao núcleo.
 - (f) Geralmente implementado por bibliotecas.
 - (g) Se um thread bloquear, todos os demais têm de esperar por ele.
 - (h) Cada thread no nível do usuário tem sua correspondente dentro do núcleo.
 - (i) É o modelo com implementação mais complexa
16. O sistema operacional mantém uma fila de tarefas prontas. Explique o papel do escalonador e como ele pode decidir qual tarefa será executada.
17. Explique o que é escalonamento round-robin, dando um exemplo.
18. Explique o que é, para que serve e como funciona a técnica de aging.

Desafio

1. Indique quantas letras “X” serão impressas na tela pelo programa abaixo quando for executado com a seguinte linha de comando: `a.out 4 3 2 1` O comando `a.out` resulta da compilação do programa a seguir:

```
17 #include <stdio.h>
18 #include <sys/types.h>
19 #include <unistd.h>
20 #include <stdlib.h>
21
22 int main(int argc, char *argv[]) {
23     pid_t pid[11]; // espaço extra para pid[N]
24     int i;
25
26     // lê o penúltimo argumento como N
27     int N = atoi(argv[argc - 2]);
28
29     // cria N processos filhos
30     for (i = 0; i < N; i++) {
31         pid[i] = fork();
32     }
33
34     // se o primeiro e o último não forem processos filhos
35     if (pid[0] != 0 && pid[N - 1] != 0) {
36         pid[N] = fork();
37     }
38
39     printf("X");
40
41     return 0;
42 }
```